

PROJECT REPORT

AXI4-Lite UART IP Core

Design, Functional Verification, FPGA Implementation and Bitstream Generation

Project Domain	Digital Design / VLSI / FPGA / Embedded Interfaces
HDL	VHDL
Design Tool	AMD Vivado 2026.1 with XSim
Target Device	xc7a35tcpg236-1 (Artix-7 family)
Prepared by	Sai Lakshmi
Project Status	Simulation, synthesis, implementation and bitstream generation completed; physical board test not performed.

A detailed record of the design objective, architecture, development workflow, verification, implementation results and limitations.

Abstract

This project develops a custom UART peripheral with an AXI4-Lite memory-mapped slave interface, written in VHDL and built using AMD Vivado 2026.1. The purpose is to connect a processor or other AXI4-Lite master to asynchronous serial communication through programmable registers. The design includes transmit and receive paths, baud-rate generation, FIFOs, status and control registers, hardware flow control, and interrupt handling. VHDL testbenches were run in XSim. The advanced test reported successful loopback transfer of 0xA5, interrupt assertion on receive completion, flow-control behavior, and interrupt clearing using Write-1-to-Clear (W1C). Synthesis and implementation were completed for the target Artix-7 device, timing checks reported no setup or hold violations, DRC reported zero errors, and a bitstream was generated. Physical FPGA testing was not performed because a hardware board was unavailable.

1. Introduction

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication method for transferring data between digital systems. It typically uses separate transmit (TX) and receive (RX) signals and does not require a shared clock between endpoints. A processor-based FPGA system benefits from a structured way to configure and control a UART. AXI4-Lite provides a simple memory-mapped interface suitable for register-oriented peripherals.

This project combines these concepts by implementing a UART peripheral controlled through AXI4-Lite transactions. A processor can access control, status and data registers instead of directly managing UART bit timing. The design is intended as a reusable FPGA IP core and was organized in an IP repository structure.

2. Problem Identification

A basic UART can transmit and receive serial data, but a processor-based FPGA system also needs a structured interface to configure the UART, send data, read received bytes, inspect status and respond to communication events. Without a memory-mapped interface and clear status/control logic, integration into a processor subsystem is less convenient. Buffering is also useful because AXI transactions and serial data transfers operate at different rates.

Problem statement: Design and verify a reusable VHDL-based UART IP core with an AXI4-Lite slave interface, configurable baud-rate timing, buffered transmit and receive data, status/control registers, hardware flow control and interrupt handling, and complete FPGA synthesis, implementation, timing analysis and bitstream generation.

Objectives:

3. Proposed Solution and Architecture

The proposed solution is a custom AXI4-Lite UART peripheral. The AXI4-Lite interface receives register read/write requests. Configuration values are held in registers; transmit data is buffered in a TX FIFO and receive data in an RX FIFO. The transmitter serializes data onto UART TX; the receiver samples UART RX and reconstructs received data. Status logic tracks FIFO and UART conditions. Interrupt logic reports supported events, while W1C behavior clears supported status bits when software writes a one to the corresponding bit. CTS/RTS signals provide hardware flow-control support.

High-level data path:

AXI4-Lite master / processor

↓

AXI4-Lite slave interface

↓

Control, status and interrupt registers + FIFO control

↓

↑

TX FIFO → UART transmitter → TX pin

RX pin → UART receiver → RX FIFO

CTS/RTS provide flow-control signals; an interrupt output signals selected events to the system.

Main functional blocks:

- **AXI4-Lite interface:** handles memory-mapped read/write handshakes.
- **Baud-rate generator:** derives baud ticks from the clock and divider setting.
- **Transmitter:** serializes parallel data into a UART frame.
- **Receiver:** synchronizes and samples RX, reconstructing received data.
- **FIFOs:** buffer data between bus and serial timing.
- **Registers:** provide configuration and software-visible status.
- **Interrupt logic:** reports supported events and supports W1C clearing.
- **Flow control:** uses CTS/RTS signals to coordinate transfer.

4. Tools and Design Environment

AMD Vivado 2026.1: RTL project management, synthesis, implementation, timing/DRC reports and bitstream generation.

VHDL: hardware description language for UART and AXI4-Lite logic.

XSim: simulation of VHDL testbenches.

Target: xc7a35tcbg236-1, an Artix-7 family device.

Git/GitHub: source version control and remote repository hosting.

5. Design and Development Process

5.1 Project setup. Created the project directory and Vivado project, selected the target FPGA part, and organized source, testbench, constraints, IP repository, documentation, images and bitstream folders.

5.2 RTL development. Developed VHDL modules for the AXI4-Lite UART function and supporting UART logic, including baud-rate generation, TX, RX and FIFO functionality. The AXI-facing design exposes memory-mapped control and data behavior.

5.3 Simulation testbenches. Prepared VHDL testbenches for individual modules and the integrated AXI UART. XSim simulations were used to inspect serial transmission/reception and peripheral control behavior.

5.4 Advanced verification. The advanced simulation reported successful loopback transfer of 0xA5, flow-control behavior when CTS_N was high, interrupt assertion after receive completion, and clearing interrupt status through W1C.

5.5 IP packaging and block-design integration. Created an IP repository structure and component metadata. The IP was integrated into a Vivado block design with a MicroBlaze-based system, as shown in the saved block-design screenshot. This demonstrates tool-level integration, not physical board operation.

5.6 Constraints. Added a 100 MHz clock constraint for the AXI clock (10 ns period). Board-specific pin assignments and electrical configuration require the exact target board and were not finalized for physical deployment.

5.7 Synthesis. Ran Vivado synthesis to map RTL into FPGA logic resources. Synthesis completed successfully without synthesis errors, according to the recorded results.

5.8 Implementation. Ran implementation to place and route the synthesized design. Implementation completed successfully and produced timing and design-rule reports.

5.9 Timing and DRC review. Reviewed post-implementation timing and DRC. The reported timing summary showed positive worst slack and no setup or hold failing endpoints. DRC reported zero errors; remaining board-related warnings must be addressed for a specific board.

5.10 Bitstream generation. Generated the FPGA configuration bitstream after synthesis and implementation, then copied it into the project bitstream folder.

5.11 Version control. Initialized Git, committed source and project artifacts, added screenshots and pushed the main branch to GitHub.

6. Functional Verification

Simulation was used to test the logic without requiring physical FPGA hardware. The advanced testbench output recorded the following results:

Verification item	Reported outcome	Purpose
UART loopback	PASS: received 0xA5 correctly	Checks simulated TX/RX data path.
CTS flow control	PASS: TX stayed IDLE while CTS_N was HIGH	Checks tested flow-control condition.
RX interrupt	PASS: interrupt generated on RX completion	Checks event notification.
W1C clear	PASS: interrupt cleared via W1C	Checks interrupt status clearing.
Overall advanced test	ALL ADVANCED HARDWARE TESTS PASSED SUCCESSFULLY (simulation log)	Passes scenarios in that testbench.

Simulation success applies to the modeled test scenarios only; it is not a physical serial-link test.

7. FPGA Synthesis and Implementation Results

The design was synthesized and implemented for xc7a35tcpg236-1 using Vivado 2026.1. The reported implementation completed, timing constraints were met for analyzed paths, and a bitstream was generated.

Metric	Result	Interpretation
Target	xc7a35tcpg236-1	Artix-7 target selected in Vivado.
AXI clock	100 MHz / 10 ns	Clock constraint applied.
WNS / TNS	+2.492 ns / 0.000 ns	No setup failing endpoints reported.
WHS / THS	+0.026 ns / 0.000 ns	No hold failing endpoints reported.
Setup/hold failing endpoints	0 / 0	All analyzed constrained endpoints passed.
DRC errors	0	No DRC errors; board-specific warnings may remain.
Bitstream	design_1_wrapper.bit	Generated and copied to bitstream/.

Resource utilization: 1,800 Slice LUTs (8.65%), 1,613 Slice Registers (3.88%), 4 Block RAM tiles (8.00%), 0 DSPs (0%), 3 BUFGCTRL (9.38%), and 95 of 106 bonded I/O (89.62%). I/O use should be checked against the selected board's pin plan.

Power estimate: total on-chip power 0.092 W (0.022 W dynamic, 0.070 W static), junction temperature 25.5°C. The report confidence was Low because input and internal switching activity were not fully specified; this is a tool estimate, not a physical measurement.

8. Screenshots and Evidence

The repository contains screenshots in images/. The README can embed them using relative Markdown paths, for example:

```
![Synthesis Result](images/synthesis.png)
![Vivado Block Design](images/block_design.png)
![Implementation Result](images/implementation.png)
![Timing Summary](images/timing.png)
```

These images document synthesis, block design, implementation and timing results. Numerical results are also listed in this report.

9. Project Structure

AXI_UART_IP/	
■■■ rtl/	VHDL design source
■■■ tb/	VHDL simulation testbenches
■■■ constraints/	XDC constraints
■■■ ip_repo/	Packaged IP metadata and source
■■■ bitstream/	Generated FPGA bitstream
■■■ docs/	Specification and documentation
■■■ images/	Vivado screenshots
■■■ vivado/	Vivado project workspace
■■■ .gitignore	
■■■ README.md	

Generated cache, run and simulation files are excluded through .gitignore.

10. Outcomes and Learning

- Built a custom register-controlled UART peripheral with an AXI4-Lite interface in VHDL.
- Practiced UART framing, baud-rate timing, buffering, status/control behavior and bus handshakes.
- Created and ran XSim testbenches, including integrated loopback and advanced checks.
- Used synthesis, implementation, timing analysis and DRC to evaluate FPGA-tool readiness.
- Generated a bitstream and organized the project under Git version control.
- Practiced IP packaging and Vivado block-design integration.

11. Limitations and Future Work

Physical hardware testing has not been performed because a development board was unavailable. Therefore, real pin-level UART communication, board-specific electrical configuration, measured power and on-board processor/software interaction remain unverified.

Future work:

- Select the exact development board and add correct package pin assignments.
- Set board-appropriate configuration voltage properties and review remaining board-specific DRC warnings.
- Program the board and test UART TX/RX using a USB-to-UART adapter or second serial device.
- Run processor software to verify register access, baud configuration, data transfer, status and interrupts.
- Capture hardware logs/waveforms and add them as board-level evidence.
- Improve power estimation with realistic switching activity and compare with measurements when hardware is available.
- Review the register map, reset behavior, error handling and IP integration guide before broader reuse.

12. Conclusion

The project addressed the need for a processor-accessible UART peripheral by combining an AXI4-Lite slave interface with UART TX/RX logic, baud-rate generation, FIFOs, status/control functionality, flow control and interrupt handling. The design was written in VHDL, tested through XSim and integrated into a Vivado block design. The advanced simulation reported successful 0xA5 loopback, CTS flow-control, receive interrupt and W1C clearing tests. Synthesis and implementation completed, timing results showed positive WNS and WHS with no failing setup or hold endpoints, DRC reported zero errors, and a bitstream was generated. The project is complete at the RTL simulation and FPGA-tool implementation stage; physical board testing remains the main validation step.

Appendix A. Suggested viva / interview explanation

"I designed a custom AXI4-Lite UART IP core in VHDL using Vivado. The AXI4-Lite interface allows a processor to configure the UART and access transmit and receive data through memory-mapped registers. I added baud-rate generation, TX/RX buffering, status and interrupt handling, CTS/RTS flow control, and W1C interrupt clearing. I verified the design in XSim, including a 0xA5 loopback and interrupt tests, then completed synthesis, implementation, timing analysis and bitstream generation for an Artix-7 target. I have not yet tested it on a physical FPGA board."

Appendix B. Key terminology

Term	Meaning
AXI4-Lite	A lightweight memory-mapped bus protocol for register access.
UART	Asynchronous serial communication interface.
TX / RX	Transmit and receive data paths.
FIFO	First-In, First-Out buffer preserving data order.
Baud rate	Signaling rate used to time serial symbols.
CTS / RTS	Hardware flow-control signals.
Interrupt	Signal notifying a processor/controller of an event.
W1C	Write-One-to-Clear status-bit behavior.
Synthesis	Maps RTL into a technology-specific hardware representation.
Implementation	Places and routes the mapped design on FPGA resources.
Bitstream	Configuration file used to program an FPGA.

Report note: Results reflect the project logs and screenshots described during development. Unavailable measurements are explicitly identified as estimates or future validation work.